

vista de implementación

Es una vista de un modelo que contiene una declaración estática de los componentes de un sistema, de sus dependencias y posiblemente de las clases que sean implementadas por ese componente.

Véase diagrama de componentes.

vista de interacción

Se trata de la visualización de un modelo que muestra el intercambio de mensajes entre objetos para lograr algún propósito. Consta de colaboraciones e interacciones y se muestra empleando diagramas de colaboración y diagramas de secuencia.

vista de máquina de estados

Aspecto del sistema relacionado con la especificación del comportamiento de elementos individuales a lo largo de su vida. Esta vista contiene máquinas de estados. Está relativamente agrupada con otras vistas de comportamiento en la vista dinámica.

vista dinámica

Ese aspecto de un modelo que se ocupa de la especificación y de la implementación del comportamiento a lo largo del tiempo, contrapuesto a la estructura estática que se encuentra en la vista estática. La vista dinámica es un término de agrupación que incluye la vista de casos de uso, la vista de máquina de estados, la vista de actividades, y la vista de interacción.

vista estática

Vista del modelo global que caracteriza los elementos de un sistema y sus relaciones entre sí. Incluye a los clasificadores y sus relaciones: sucesos, generalización, dependencia y realización. A veces recibe el nombre de *vista de clases*.

Semántica

La vista estática muestra la estructura estática de un sistema y en particular las clases de elementos que existen (tales como las clases y tipos), su estructura interna y sus relaciones con otros elementos. Las vistas estáticas no muestran información temporal, aunque puede contener apariciones materializadas de cosas que tengan o describan un comportamiento temporal, tal como las especificaciones de operaciones o de eventos.

Entre los componentes de más alto nivel de una vista estática se cuentan los clasificadores (clase, interfaz, tipo de datos), las relaciones (asociación, generalización, dependencia, reali-

zación), las restricciones y los comentarios. También contiene paquetes y subsistemas como unidades organizativas. Los demás componentes están subordinados y contenidos en los elementos de alto nivel.

Las vistas de implementación, de despliegue y de administración del modelo están relacionadas con la vista estática y suelen combinarse con ella en otros diagramas.

La vista estática se puede contrastar con la vista dinámica, que la complementa y se basa en ella.

vista estructural

Vista global de un modelo que hace hincapié en la estructura de los objetos del sistema, incluyendo sus tipos, clases, relaciones, atributos y operaciones.

vista funcional

Una vista que se ocupa de la descomposición de un sistema en las funciones o las operaciones que proporcionan su funcionalidad. Una vista funcional no se considera normalmente orientada a objetos y puede conducir a una arquitectura difícil de mantener.

En los métodos de desarrollo tradicionales, el diagrama de flujo de datos es el corazón de la vista funcional. UML no apoya directamente una visión funcional, aunque los grafos de actividades tienen algunas características funcionales.

Los elementos estándar son palabras clave predefinidas que corresponden a restricciones, estereotipos y etiquetas. Representan conceptos de utilidad general que no son suficientemente significativos o suficientemente diferentes de los conceptos centrales para incluirlos como conceptos centrales de UML. Tienen la misma relación con los conceptos centrales de UML que la que posee una biblioteca de subrutinas con un lenguaje de programación. No forman parte del lenguaje en sí, pero forman parte del entorno con que puede contar el usuario cuando hace uso del lenguaje. La lista incluye también palabras clave de notación —son palabras reservadas que aparecen en el símbolo de otro elemento del modelo pero que denotan elementos incorporados del modelo, no estereotipos—. Para las palabras clave se enumera el símbolo de la notación.

Las referencias cruzadas aluden a artículos del Capítulo 13, Enciclopedia de Términos.

access

(estereotipo de la dependencia Permiso)

Dependencia estereotipada entre dos paquetes que denota que el contenido público del paquete destino es visible para el espacio de nombres del paquete origen.

Véase acceso.

association

(estereotipo de ExtremoAsociación)

Restricción aplicada al extremo de una asociación (incluyendo los extremos de un enlace o un extremo de un rol de asociación), que especifica que la instancia correspondiente es visible a través de una asociación virtual y no a través de un enlace transitorio, tal como un parámetro o variable local.

Véase asociación, extremo de asociación, rol de asociación.

become

(estereotipo de la relación Flujo)

Dependencia estereotipada cuyo origen y destino representan la misma instancia en distintos instantes de tiempo, pero pudiendo tener cada una valores, instancia de estado y roles diferentes.

Una dependencia de conversión que va desde **A** hasta **B** significa que la instancia **A** se convierte en **B** con (posiblemente) nuevos valores, instancia de estado y roles. La notación de conversión es una flecha discontinua que va del origen al destino con la palabra clave «**become**».

Véase become en la enciclopedia de términos.

bind

(palabra reservada en un símbolo de dependencia)

Palabra reservada de dependencia que denota una relación de ligadora. Va seguida por una lista de argumentos separados por comas y encerrada entre paréntesis.

Véase enlace, elemento ligado, plantilla.

call

(estereotipo de la dependencia Uso)

Dependencia estereotipada cuyo origen es una operación y cuyo destino es una operación. Una dependencia de llamada específica que el origen invoca a la operación destino. Una dependencia de llamada puede conectar una operación origen con cualquier operación destino situada a su alcance, incluyendo (pero sin tener como límite) las operaciones del clasificador que la contiene y las operaciones de otros clasificadores visibles.

Véase llamada, uso.

complete

(restricción de Generalización)

Restricción que se aplica a un conjunto de generalizaciones, para especificar que se han especificado todos los hijos (aunque se haya omitido alguno) y que no se espera declarar hijos adicionales posteriormente.

Véase generalización.

copy

(estereotipo de la relación Flujo)

Relación de flujo estereotipada cuyo origen y destino son instancias distintas, pero teniendo ambas los mismos valores, instancia de estado y roles (aunque con distinta identidad). Una dependencia de copia de **A** hacia **B** significa que **B** es una copia exacta de **A**. Los futuros

cambios de A no se reflejan necesariamente en B. La notación de copia es una flecha discontinua que va del origen al destino con la palabra clave «**copy**».

Véase acceder, copia.

create

(estereotipo de característica de comportamiento)

Característica de comportamiento estereotipada que denota que la característica indicada crea una instancia del clasificador al que está asociada la característica.

(estereotipo de evento)

Evento estereotipado que denota que la instancia que encierra la máquina de estados a que se aplica el evento se crea. La creación se puede aplicar únicamente a una transición inicial situada en el nivel más elevado de la máquina de estados. De hecho, éste es el único tipo de disparador que se puede aplicar a una transición inicial.

(estereotipo de la dependencia Utilización)

Dependencia estereotipada que denota que el clasificador cliente crea instancias del clasificador proveedor.

Véase creación, uso.

derive

(estereotipo de la dependencia Abstracción)

Dependencia estereotipada cuyo origen y destino suelen ser (aunque no necesariamente) elementos del mismo tipo. Una dependencia de derivación específica que el origen se puede computar a partir del destino. El origen puede haberse implementado por razones de diseño, tales como la eficiencia, aun cuando sea redundante desde un punto de vista lógico.

Véase derivación, elemento derivado.

destroy

(estereotipo de característica de comportamiento)

Característica de comportamiento estereotipada que denota que la característica indicada destruye una instancia del clasificador al que está asociada la característica.

(estereotipo de evento)

Evento estereotipado que denota la destrucción de la instancia que contiene a la máquina de estados a la que se aplica ese tipo de evento.

Véase destrucción.

destroyed

(restricción de Rol de clasificador y de Rol de asociación)

Denota que existe una instancia de ese rol al principio de la ejecución de la interacción que lo contiene pero esa instancia es destruida antes de finalizar la ejecución.

Véase rol de asociación, rol de clasificador, colaboración, destrucción.

disjoint

(restricción de la Generalización)

Restricción que se aplica a un conjunto de generalizaciones, especificando que un objeto no puede ser instancia de más de un hijo en ese conjunto de generalizaciones. Esta situación únicamente surgiría con herencia múltiple.

Véase generalización.

document

(estereotipo de componente)

Componente estereotipado que representa un documento.

Véase componente.

documentation

(etiqueta de un elemento)

Comentario, descripción o explicación del elemento al que está asociada.

Véase comentario, cadena.

enumeration

(palabra clave de los símbolos de clasificador)

Palabra reservada de un tipo de datos de enumeración, cuyos detalles especifican un dominio que consta de un conjunto de identificadores que son los posibles valores de una instancia de ese tipo de datos.

Véase enumeración.

executable

(estereotipo de componente)

Componente estereotipado que denota un programa que se puede ejecutar en un nodo.

Véase componente.

extend

(palabra reservada del símbolo de Dependencia)

Palabra clave de un símbolo de dependencia que denota una relación de extensión entre casos de uso.

Véase extender.

facade

(estereotipo de paquete)

Paquete estereotipado que contiene únicamente referencias a elementos del modelo que son propiedad de otro paquete. Se emplea para proporcionar una vista pública de cierta parte del contenido de un paquete. Una fachada no contiene ningún elemento del modelo que sea propio de ella.

Véase paquete.

file

(estereotipo de componente)

Archivo es un componente estereotipado que representa un documento que contiene código fuente o bien datos.

Véase componente.

framework

(estereotipo de Paquete)

Paquete estereotipado que consta principalmente de patrones.

Véase paquete.

friend

(estereotipo de dependencia de permiso)

Dependencia estereotipada cuyo origen es un elemento del modelo tal como una operación, clase o paquete y cuyo destino es un elemento diferente del modelo, tal como una clase o

paquete. La relación de amistad concede al destino acceder al origen, independientemente de la visibilidad declarada. Extiende la visibilidad del origen de tal modo que el destino puede ver el interior del origen.

Véase acceso, amigo, visibilidad.

global

(estereotipo de ExtremoAsociacion)

Restricción que se aplica a un extremo de asociación (incluyendo los extremos de enlace y los extremos de roles de asociación), especificando que el objeto asociado es visible porque se encuentra en un alcance global respecto al objeto que está en el otro extremo del enlace.

Véase asociación, extremo de asociación, colaboración.

implementation

(estereotipo de generalización)

Generalización estereotipada que denota que el cliente hereda la lista implementación del proveedor (sus atributos, operaciones y métodos) pero no hace públicas las interfaces del proveedor ni garantiza que va a admitirlas, violando por tanto la capacidad de sustitución. Es una herencia privada.

Véase generalización, herencia privada.

implementationClass

(estereotipo de clase)

Clase estereotipada que no es un tipo y que representa una implementación de una clase en algún lenguaje de programación. Un objeto puede ser instancia de un máximo de una clase de implementación. Por contraste, un objeto puede ser una instancia de múltiples clases ordinarias en un instante y perder o ganar clases con el paso del tiempo. Una instancia de una clase de implementación también puede ser instancia de cero o más tipos.

Véase clase de implementación, tipo.

implicit

(estereotipo de asociación)

Estereotipo de una asociación, que especifica que la asociación no es manifiesta (no está implementada) sino sólo conceptual.

Véase asociación.

import

(estereotipo de dependencia de permiso)

Dependencia estereotipada entre dos paquetes, que denota que el contenido público del paquete destino se añade al espacio de nombres del paquete origen.

Véase acceso, importar.

include

(palabra clave de símbolo de dependencia)

Palabra clave de los símbolos de dependencia que denota una relación de inclusión entre casos de uso.

Véase incluir.

incomplete

(restricción de generalización)

Restricción que se aplica a un conjunto de generalizaciones, especificando que no se han especificado todos los hijos y que se espera la adición de otros hijos.

Véase generalización.

instanceOf

(palabra clave de símbolo de dependencia)

Una metarelación cuyo cliente es una instancia y cuyo proveedor es un clasificador. Una dependencia **instanceOf** que va desde **A** hacia **B** indica que **A** es una instancia de **B**. La notación de **instanceOf** es una flecha discontinua con la palabra clave «**instanceOf**».

Véase descriptor, instancia, instancia de.

instantiate

(estereotipo de dependencia de uso)

Dependencia estereotipada entre clasificadores, que indica que las operaciones que afectan al cliente crean instancias del proveedor.

Véase instanciación, uso.

invariant

(estereotipo de restricción)

Restricción estereotipada que es preciso asociar a un conjunto de clasificadores o relaciones. Denota las condiciones que debe imponer la restricción a efectos de los clasificadores o relaciones y sus instancias.

Véase invariante.

leaf

(palabra reservada de ElementoGeneralizable y CaracterísticaDeComportamiento)

Indica un elemento que no puede tener descendientes o no se pueden anular, esto es, que no es polimórfico.

Véase hoja, polimórfico/a.

library

(estereotipo de componente)

Componente estereotipado que representa una biblioteca estática o dinámica.

Véase componente.

local

(estereotipo de ExtremoAsociación)

Estereotipo de un extremo de asociación, extremo de enlace o extremo de rol de asociación, que especifica que el objeto asociado se encuentra en el alcance local del objeto que está en el otro extremo.

Véase asociación, extremo de asociación, colaboración, enlace transitorio.

location

(etiqueta de símbolo clasificador)

El componente que sirve de base al clasificador.

(palabra reservada de símbolo de instancia de componente)

Instancia de nodo en la que reside la instancia de componente.

Véase componente, localización, nodo.

metaclass

(estereotipo de clasificador)

Clasificador estereotipado que denota que la clase es una metaclasses de alguna otra clase.

Véase metaclasses.

new

(restricción de RolClasificador y Roldeasociación)

Denota que se crea una instancia del rol durante la ejecución de la interacción que lo contiene y que esa instancia sigue existiendo al finalizar la ejecución.

Véase rol de asociación, rol de clasificador, colaboración, creación.

overlapping

(restricción de generalización)

Restricción aplicada a un conjunto de generalizaciones, que especifica que un objeto puede ser instancia de más de un hijo dentro de ese conjunto de generalizaciones. Sólo puede surgir esta situación con herencia múltiple o clasificación múltiple.

Véase generalización.

parameter

(estereotipo de ExtremoAsociación)

Estereotipo de un extremo de asociación (incluyendo el extremo de un enlace y el extremo de un rol de asociación), que especifica que un objeto asociado es un argumento de una llamada a una operación del objeto situado en el otro extremo.

Véase rol de asociación, rol de clasificador, colaboración, parámetro, enlace transitorio.

persistence

(etiqueta de clasificador, asociación y atributo)

Denota si el valor de una instancia debe o no sobrevivir al proceso que lo ha creado. Los valores son persistentes y transitorios. Si se usa en un atributo, permite una mejor discriminación acerca de cuáles de los valores de los atributos deberían mantenerse dentro de un clasificador.

Véase objeto persistente.

postcondition

(estereotipo de restricción)

Restricción estereotipada que debe estar asociada a una operación. Denota las condiciones que deben cumplirse después de haber invocado a la operación.

Véase postcondición.

powertype

(estereotipo de clasificador)

Clasificador estereotipado que denota que el clasificador es una metaclassa cuyas instancias son subclases de otra clase.

(palabra reservada de símbolo de dependencia)

Relación cuyo cliente es un conjunto de generalizaciones y cuyo proveedor es un supratipo. El proveedor es un supratipo del cliente.

Véase supratipo.

precondition

(estereotipo de restricción)

Restricción estereotipada que debe estar asociada a una operación. Denota las condiciones que deben cumplirse en el momento de invocar a la operación.

Véase precondition.

process

(estereotipo de clasificador)

Clasificador estereotipado que es una clase activa que representa a un proceso pesado.

Véase clase activa, proceso, hilo.

refine

(estereotipo de dependencia de abstracción)

Estereotipo de dependencia que denota una relación de refinamiento.

Véase refinamiento.

requirement

(estereotipo de comentario)

Comentario estereotipado que enuncia una responsabilidad u obligación.

Véase requisito, responsabilidad.

responsibility

(estereotipo de comentario)

Contrato u obligación del clasificador. Se expresa mediante una cadena de texto.

Véase responsabilidad.

self

(estereotipo de ExtremoAsociación)

Estereotipo de un extremo de asociación (incluyendo los extremos de enlace y los extremos de rol de asociación), que especifica un pseudoenlace de un objeto consigo mismo con el propósito de invocar a una operación del mismo objeto dentro de una interacción. No implica una estructura de datos real.

Véase rol de asociación, rol de clasificador, colaboración, parámetro, enlace transitorio.

semantics

(etiqueta de clasificador)

Especificación del significado del clasificador.

(etiqueta de operación)

Especificación del significado de la operación.

Véase semántica.

send

(estereotipo de dependencia de uso)

Dependencia estereotipada cuyo cliente es una operación o clasificador y cuyo proveedor es una señal, que especifica que el cliente envía la señal a algún destino no especificado.

Véase enviar, señal.

stereotype

(palabra clave de símbolo clasificador)

Palabra clave para la definición de un estereotipo. Se puede utilizar el nombre como nombre de estereotipo en otros elementos del modelo.

Véase estereotipo.

stub

(estereotipo de paquete)

Paquete estereotipado que representa un paquete que proporciona las partes públicas de otro paquete, pero nada más.

Observe que la palabra también se usa en UML para describir las transiciones abreviadas.

Véase paquete.

system

(estereotipo de paquete)

Paquete estereotipado que contiene un conjunto de modelos de un sistema, describiéndolo desde distintos puntos de vista no necesariamente disjuntos —es la estructura de más alto nivel en la especificación del sistema—. También contiene relaciones entre elementos del modelo pertenecientes a diferentes modelos. Estas relaciones y restricciones no añaden información semántica a los modelos. Lo que hacen es describir las relaciones de los modelos en sí; por ejemplo para el seguimiento de requisitos y de la historia del desarrollo.

Un sistema puede ser realizado por un conjunto de sistemas subordinados, cada uno de los cuales es descrito por su propio conjunto de modelos reunidos en un paquete de sistema por separado. Un paquete de sistema sólo puede estar contenido en un paquete de sistema.

Véase paquete, modelo, sistema.

table

(estereotipo de componente)

Componente estereotipado que representa una tabla de una base de datos.

Véase componente.

thread

(estereotipo de clasificador)

Clasificador estereotipado que es una clase activa y representa un flujo de control ligero.

Observe que en este libro se utiliza la palabra con un sentido más amplio, denotando cualquier centro de ejecución independiente y concurrente.

Véase clase activa, hilo.

trace

(palabra clave de dependencia de abstracción)

Palabra clave de un símbolo de dependencia que denota una relación de traza.

Véase traza.

transient

(restricción de rol de clasificador y rol de asociación)

Afirma que se crea una instancia del rol durante la ejecución de la interacción que lo contiene, pero esa instancia se destruye antes de haber finalizado la ejecución.

Véase rol de asociación, rol de clasificador, colaboración, creación, destrucción, enlace transitorio.

type

(estereotipo de clase)

Clase estereotipada que se emplea para la especificación de un dominio de instancias (objetos) junto con las operaciones que son aplicables a esos objetos. Un tipo no puede contener métodos pero puede poseer atributos y asociaciones.

Véase clase de implementación, tipo.

use

(palabra clave de símbolo de dependencia)

Palabra clave de dependencia que denota una relación de uso.

Véase uso.

utility

(estereotipo de clasificador)

Clasificador estereotipado que carece de instancias. Describe una colección con nombre de atributos y operaciones que no son miembros, todos los cuales tienen alcance de clase.

Véase utilidad.

xor

(restricción de asociación)

Restricción aplicada a un conjunto de asociación que comparten una conexión con una clase; especifica que todo objeto de la clase compartida tendrá enlaces procedentes de una sola de las asociaciones. Se trata de una restricción o-exclusivo (no o-inclusivo).

Véase asociación.

Parte 4: Apéndices



Apéndice A

Metamodelo de UML



Documentos de definición de UML

UML está definido en un conjunto de documentos publicados por el Object Management Group [UML-98]. Estos documentos se han incluido en el CD que acompaña al libro. Este capítulo explica la estructura del modelo semántico de UML que se describe en los documentos.

El UML se define formalmente empleando un metamodelo —esto es, un modelo de las estructuras de UML. El metamodelo, a su vez, se expresa en UML. Esto es un ejemplo de intérprete metacircular— esto es, un lenguaje definido en términos de sí mismo. Pero las cosas no son completamente circulares. Sólo se emplea un pequeño subconjunto de UML para definir el metamodelo. En principio, el punto fijo de la definición se podría basar en alguna definición más básica. En la práctica, no es necesario llegar a estos extremos.

Cada sección del documento semántico contiene un diagrama de clases que muestra una porción del metamodelo; una descripción textual de las clases del metamodelo definidas en esa sección, con sus atributos y relaciones; una lista de restricciones aplicables a los elementos expresadas en lenguaje natural y en OCL y por último una descripción textual de la semántica dinámica de las estructuras de UML definidas en la sección. Por tanto, la semántica dinámica es informal, pero una descripción completamente formal sería a la vez poco práctica y prácticamente ilegible para casi todos.

La notación se describe en otro capítulo que hace referencia al capítulo de semántica y establece la correspondencia entre los símbolos y las clases del metamodelo.

Estructura del metamodelo

El metamodelo está dividido en tres paquetes principales (Figura A1).

- El paquete de fundamentos define la estructura estática de UML.
- El paquete de elementos de comportamiento define la estructura dinámica de UML.
- El paquete de administración del modelo define la estructura organizativa de los modelos de UML.

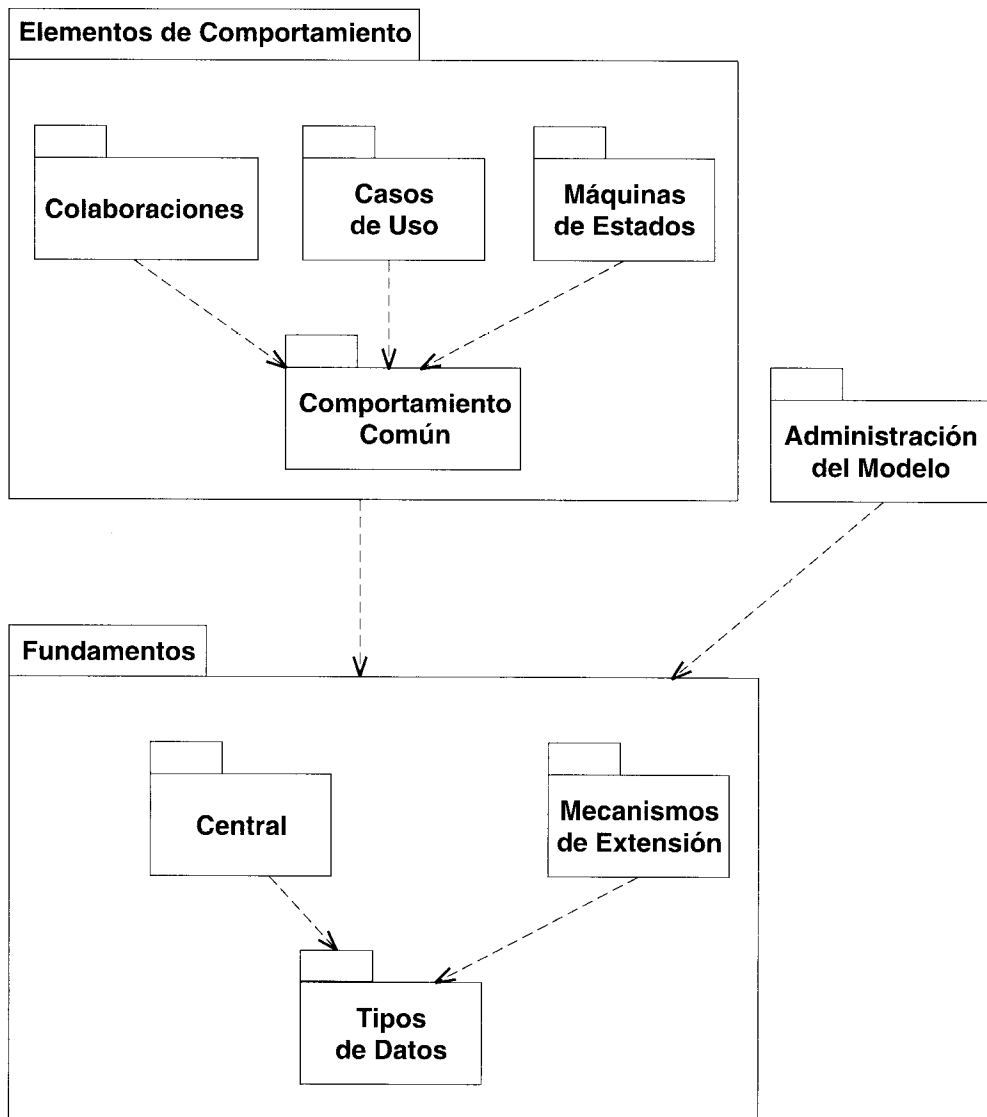


Figura A-1 Estructura de paquetes del metamodelo de UML

Paquete de fundamentos

El paquete de fundamentos contiene cuatro fundamentos.

Núcleo

El paquete núcleo describe las principales estructuras estáticas de UML. Entre ellas se cuentan los clasificadores, su contenido y sus relaciones. El contenido incluye atributo, operación, método y parámetro. Entre las relaciones se cuentan generalización, asociación y dependencia. También se definen varias metaclases abstractas tales como elemento generalizable, espacio de

nombres y elemento del modelo. El paquete también define plantilla y distintos tipos de subclases de dependencia así como componente, nodo y comentario.

Tipos de datos

El paquete de tipos de datos describe las clases de tipos de datos que se utilizan en el meta-modelo.

Mecanismos de extensión

El paquete de mecanismos de extensión describe los mecanismos restricción, estereotipo y valor etiquetado.

Paquete de elementos de comportamiento

El paquete de comportamiento tiene un subpaquete para cada vista principal y además un paquete para estructuras de comportamiento que comparten las tres vistas principales.

Comportamiento común

El paquete de comportamiento común describe señal, operación y acción. También describe instancias de clases correspondientes a diferentes descriptores.

Colaboraciones

El paquete colaboraciones describe colaboración, interacción, mensaje, rol de clasificador y asociación.

Casos de uso

El paquete casos de uso describe actor y caso de uso.

Máquinas de estados

El paquete máquinas de estados describe la estructura de una máquina de estados, incluyendo estado y distintas clases de pseudoestado, evento, señal, transición y condición de guarda. También describe estructuras adicionales para modelos de actividad tales como estado de acción, estado de actividad y estado de flujo de objeto.

Paquete de administración de modelos

El paquete de administración de modelos describe los paquetes, modelos y subsistemas. También describe las propiedades de propiedad y visibilidad de los espacios de nombres y de los paquetes. No posee subpaquetes.

Apéndice B

Resumen de la notación



Este capítulo contiene un breve resumen visual de la notación. Se han incluido los elementos principales de la notación, pero no constan todas las versiones u opciones. Para estudiar todos los detalles, consulte la entrada de cada elemento en la enciclopedia.

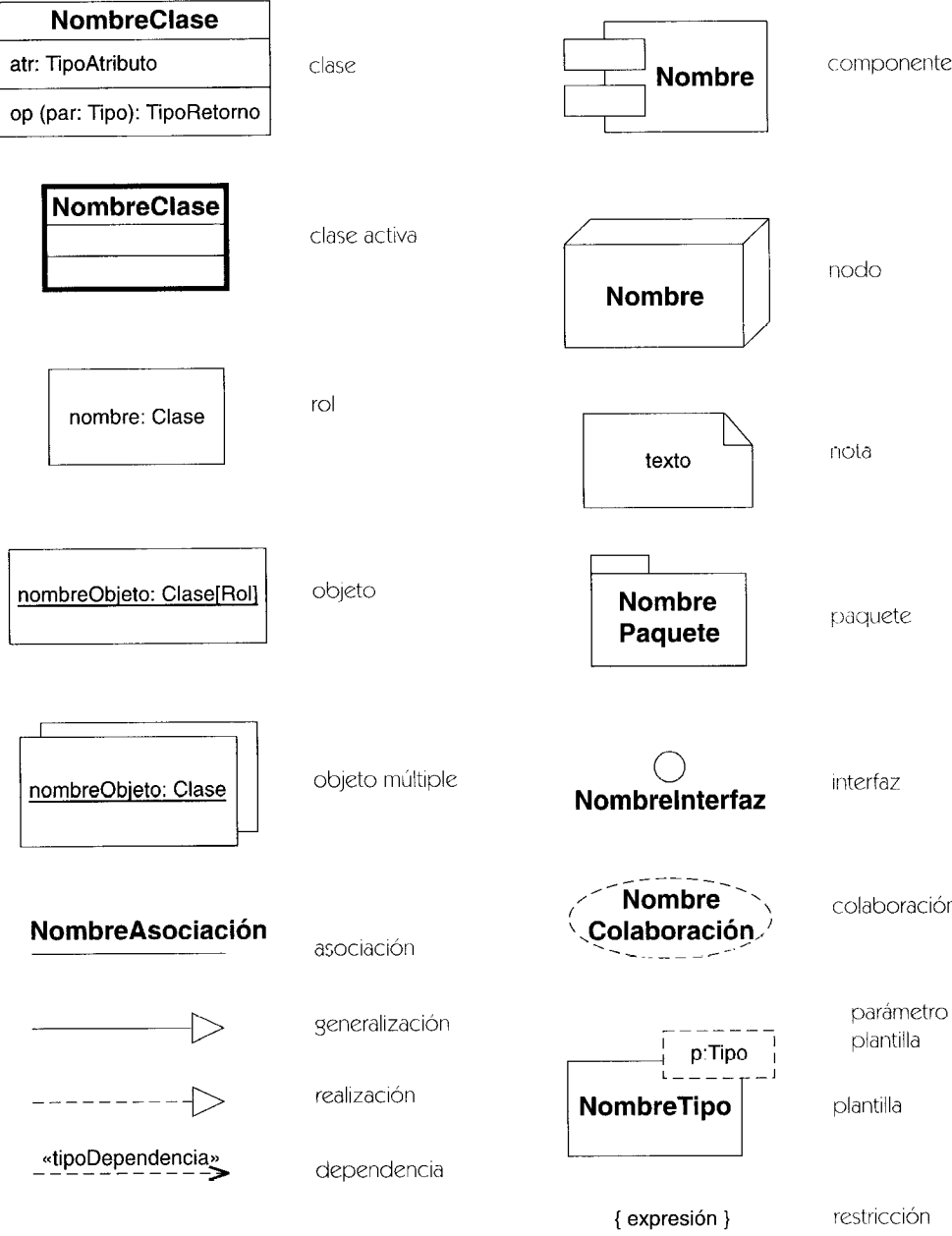


Figura B-1 Iconos de los diagramas de clase, componente, despliegue y colaboración

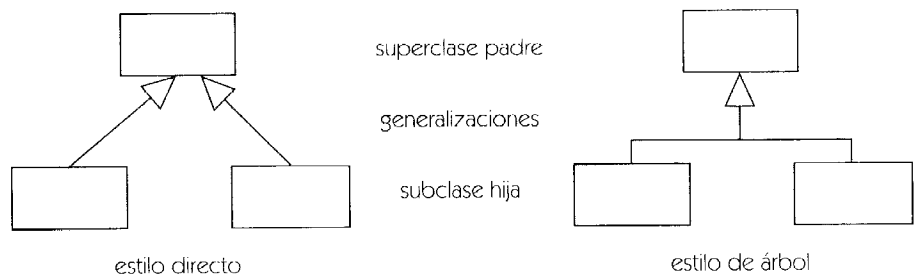


Figura B-4 Generalización

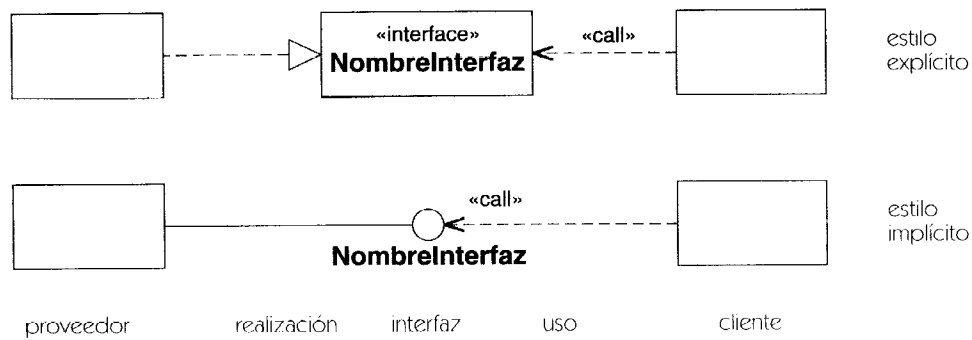


Figura B-5 Realización de una interfaz

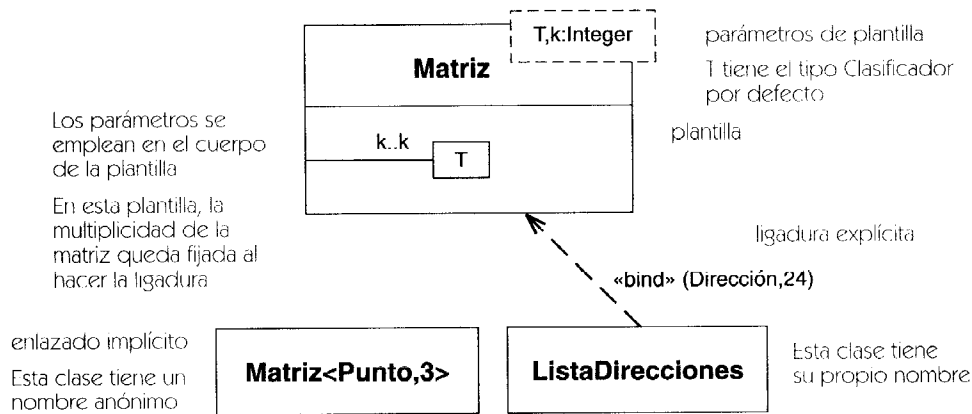


Figura B-6 Plantilla

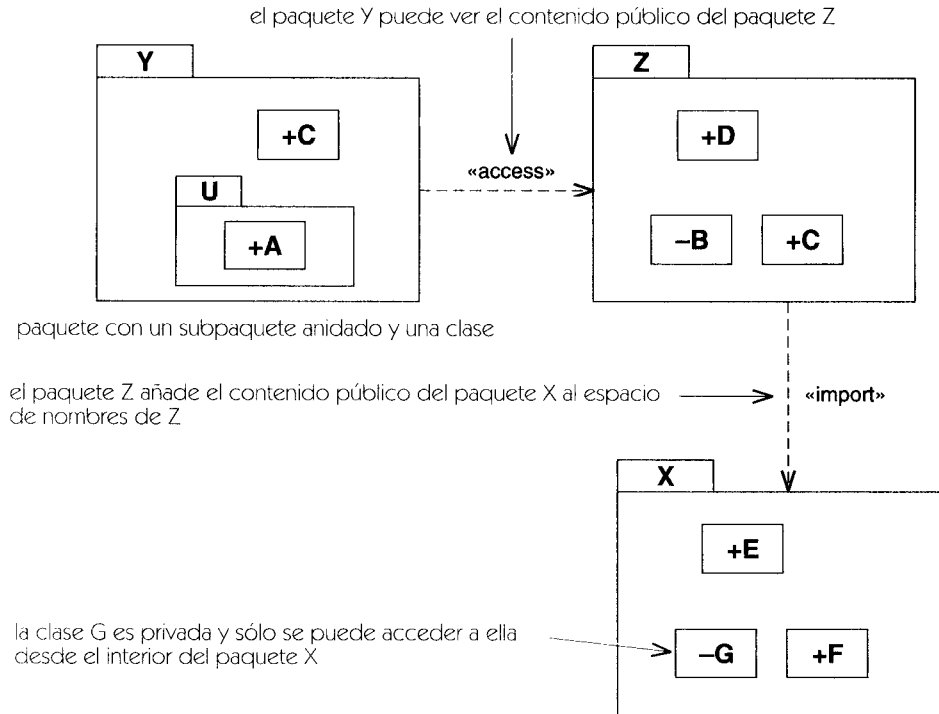


Figura B-7 Notación de paquetes

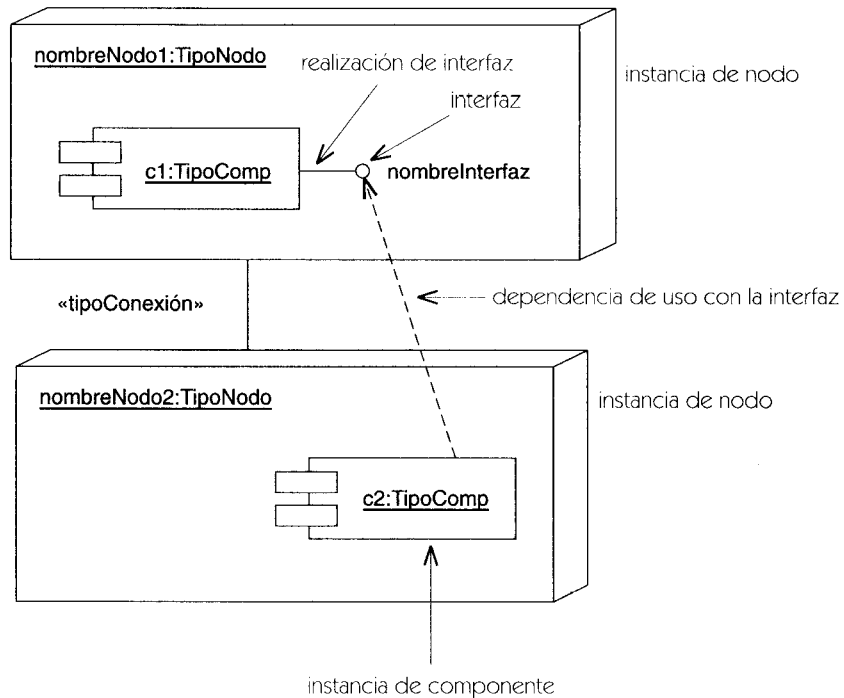


Figura B-8 Notación de componentes y nodos

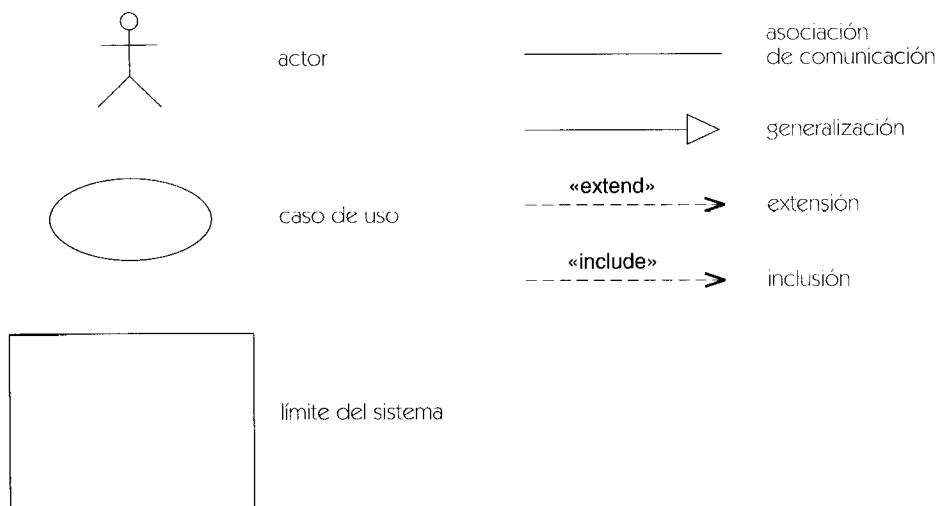


Figura B-9 Iconos de los diagramas de caso de uso

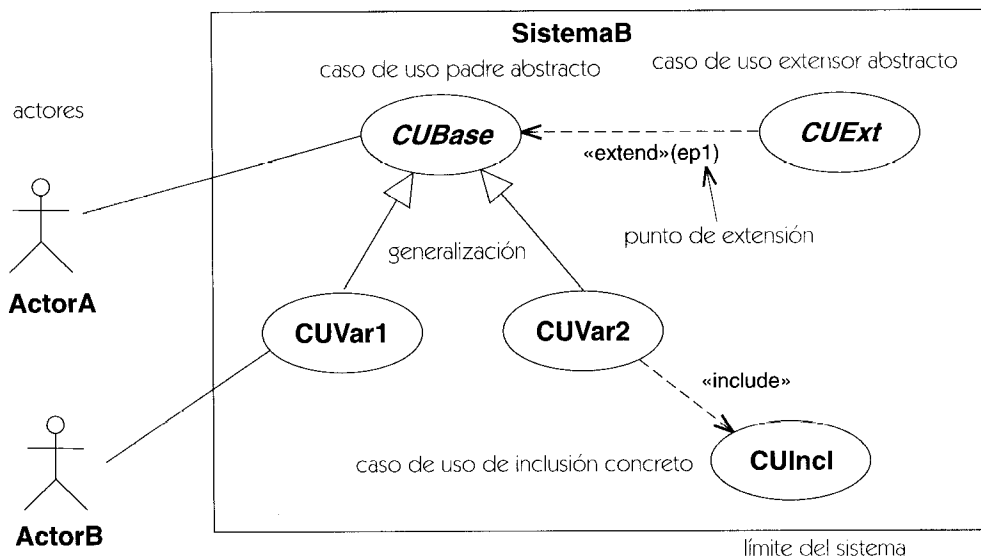


Figura B-10 Notación de los diagramas de caso de uso

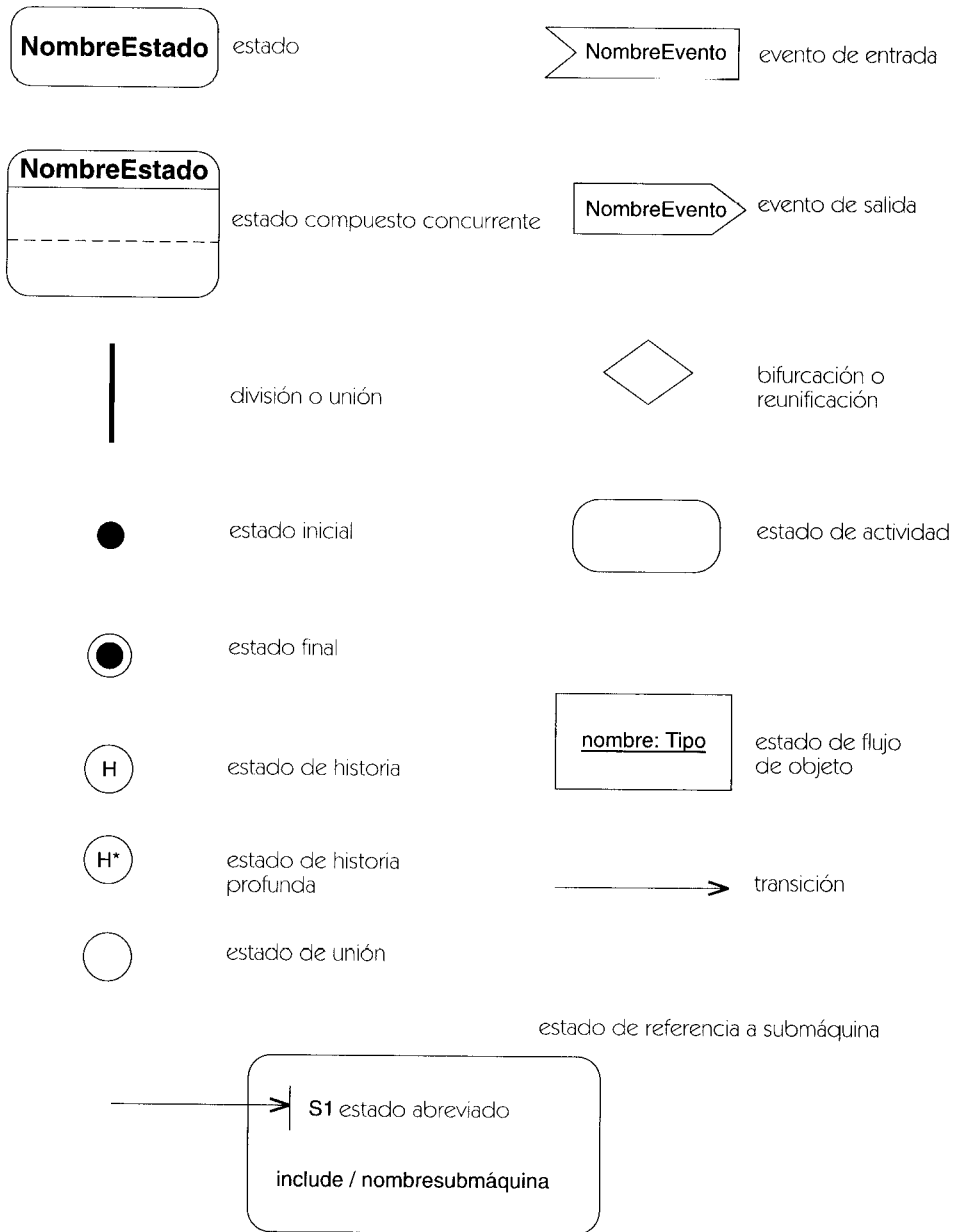


Figura B-11 Iconos de un diagrama de estados y diagramas de actividad

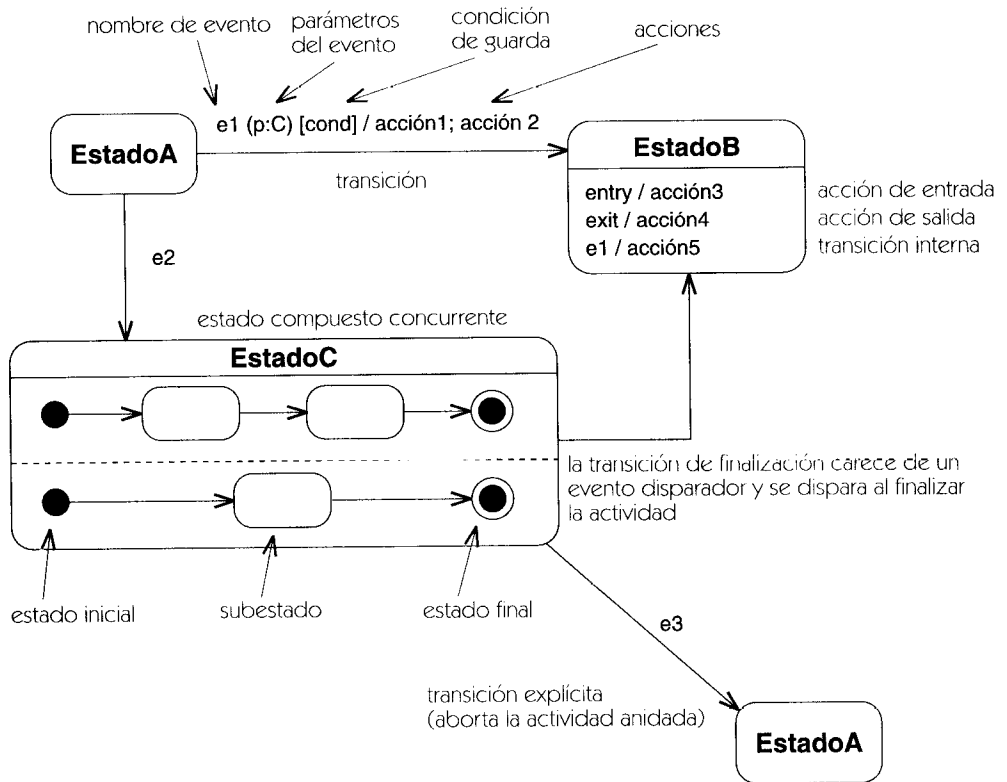
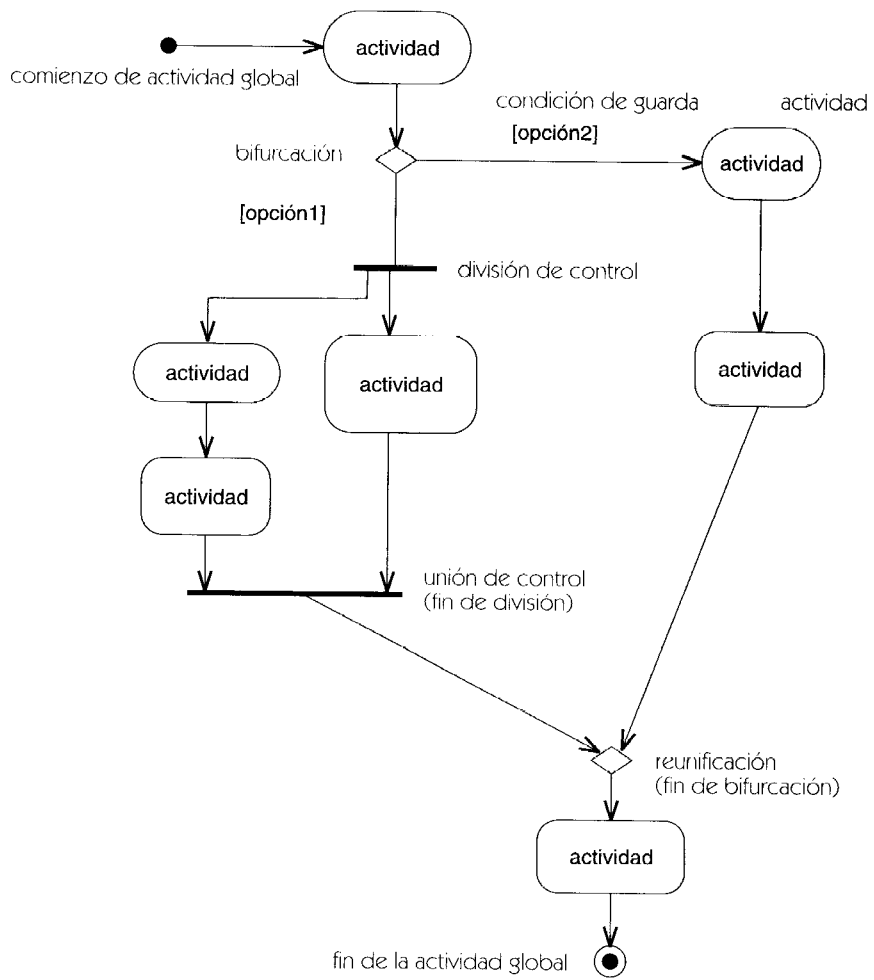


Figura B-12 Notación de diagramas de estados

NombreClase::Nombreoperación**Figura B-13** Notación de los diagramas de actividad

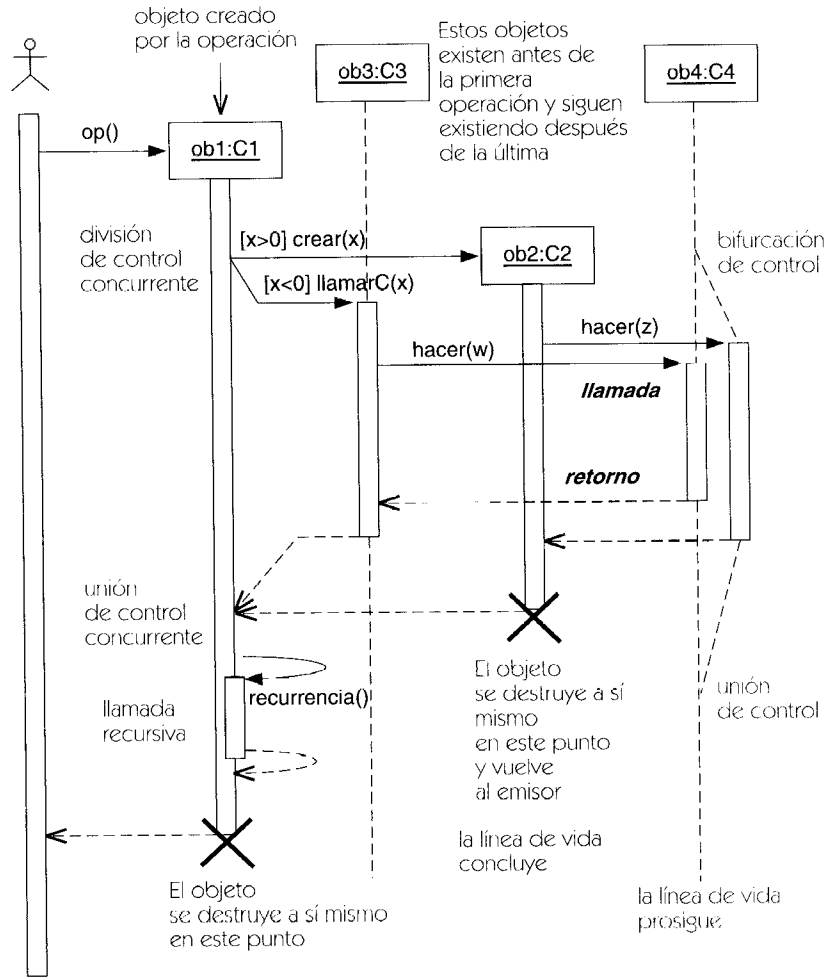


Figura B-14 Notación de diagramas de secuencia

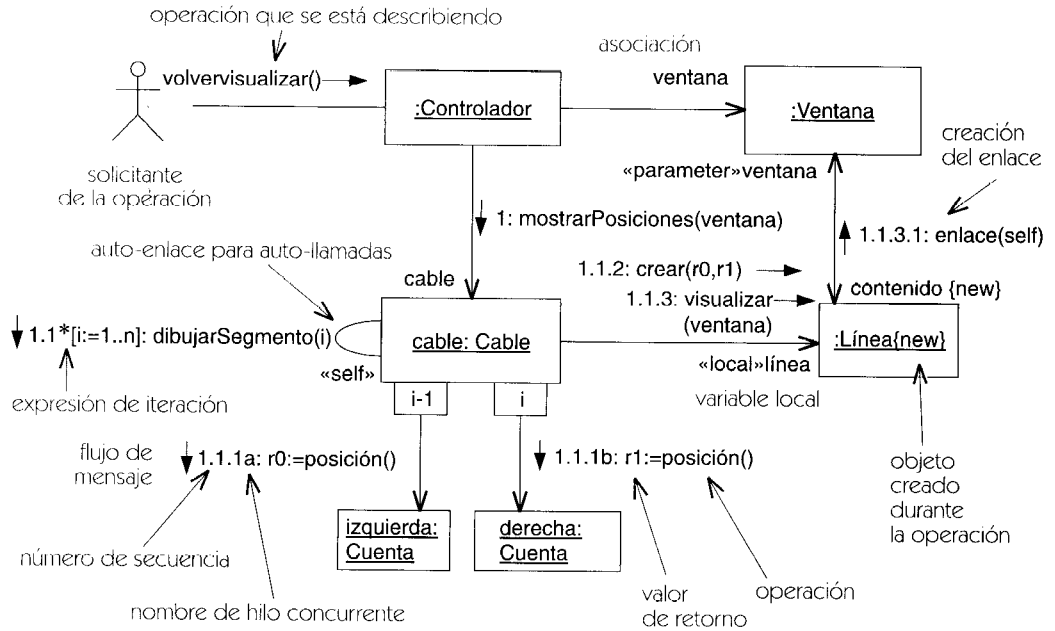


Figura B-15 Notación de diagrama de colaboración

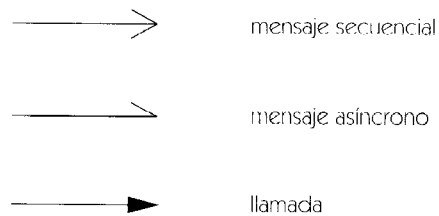


Figura B-16 Notación de mensajes

Apéndice C

Extensiones del proceso



Cómo personalizar UML

UML está diseñado para ser utilizable con múltiples propósitos. Aunque UML es un lenguaje universal que puede expresar un elevado número de propiedades fundamentales de modelado, existen ocasiones en que resulta deseable una jerga hecha a la medida de un determinado dominio de conocimiento. En los lenguajes naturales, la jerga, las germanías y los vocabularios especializados suelen facilitar la comunicación dentro de un arte o disciplina especializado, pero impiden la comunicación con los extraños. UML se puede personalizar de forma similar, empleando distintos mecanismos tales como convenciones de denominación, reglas de estilo, clases predefinidas y correspondencias implícitas con el software. En particular, se puede definir una extensión de UML empleando estereotipos, restricciones y valores etiquetados predefinidos.

UML posee muchas estructuras expresivas así que debería evitarse una extensión especializada a no ser que sea absolutamente necesaria. Por definición, una extensión sirve tan sólo a una comunidad limitada y puede dar lugar a malentendidos y confusiones por parte de los extraños. Sin embargo, si una comunidad destino está muy centrada, la potencia expresiva de una extensión puede compensar a veces la pérdida de uniformidad. Con el tiempo, algunas extensiones pueden ser tan útiles que acabarán por ser añadidas a UML; otras pueden caer en desuso.

Este capítulo describe dos extensiones que se adjuntan con los documentos de UML. Su utilización no es obligatoria y ni siquiera se recomienda necesariamente, pero indican la forma en que se puede definir una extensión para un proceso de desarrollo. En la literatura se ha descrito un cierto número de extensiones similares.

Extensiones del proceso de desarrollo de software

Estas extensiones están basadas en el proceso de desarrollo Objectory, que es un precursor del Proceso Unificado. Están destinadas a ser utilizadas en un proceso de desarrollo de software que consta de cuatro fases: captura de casos de uso, análisis, diseño e implementación.

Estas extensiones incluyen estereotipo de unidades de empaquetado, clases y asociación así como algunas restricciones relativas a la conexión de elementos. No hay etiquetas nuevas.

Estereotipos organizativos

La Tabla C-1 muestra los estereotipos organizativos, esto es, los estereotipos de modelo, clase y subsistema. Hay varios estereotipos aplicables a cada fase del desarrollo. En el proceso Objectory, cada fase posee un modelo distinto, con relaciones de traza entre los elementos de los diferentes modelos. Avanzando desde arriba hacia abajo, los términos sistema, subsistema y paquete de servicio describen capas de empaquetado.

Los estereotipos organizativos no tienen iconos especiales. Se representan mediante iconos de carpeta con el nombre del estereotipo entre comillas, tal y como aparece en la Figura C-1.

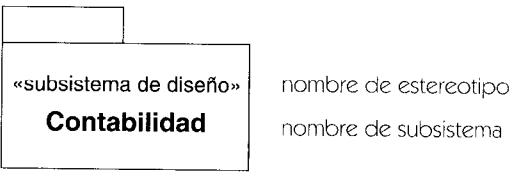


Figura C-1 Notación organizativa de estereotipos para el proceso de desarrollo de software

Tabla C-1 Estereotipos organizativos para el proceso de desarrollo de software

Clase Base	Estereotipos por Fase del Proceso			
	Caso de Uso	Análisis	Diseño	Implementación
modelo	modelo de caso de uso	modelo de análisis	modelo de diseño	modelo de implementación
paquete	sistema de caso de uso paquete de caso de uso			sistema de implementación subsistema de implementación
subsistema		sistema de análisis subsistema de análisis paquete de servicio	sistema de diseño subsistema de diseño paquete de servicio	

Estereotipos de clase

En las clases están definidos tres estereotipos: control, frontera y entidad. Se muestran en la Figura C-2.

Una *clase de control* describe objetos que administran interacciones, tales como un administrador de transiciones, un controlador de dispositivos o un monitor de un sistema operativo.

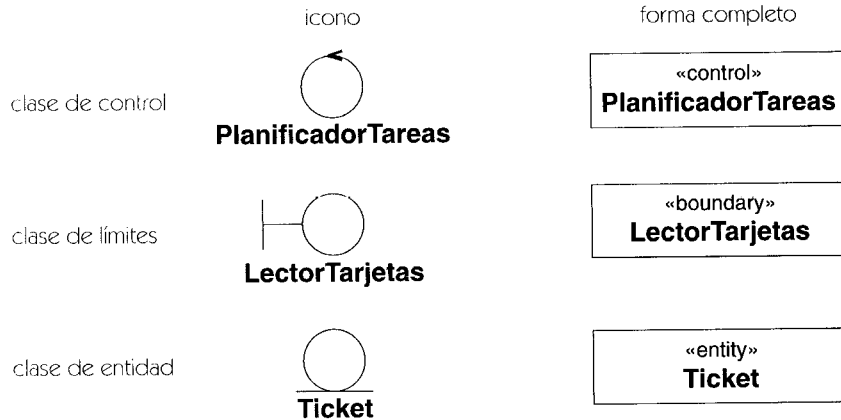


Figura C-2 Estereotipo de clase para el proceso de desarrollo de software

Tiene un comportamiento específico de un caso de uso. Normalmente, las clases de control no sobreviven a los casos de uso que admiten. Las clases de control se muestran como un círculo que tiene una punta de flecha.

Las *clases de frontera* describen objetos que median entre el sistema y los actores externos, tales como un formulario para hacer pedidos o un sensor. Se muestran como un círculo que tiene asociado un segmento en forma de T. Los objetos de frontera suelen existir durante toda la vida del sistema.

Las *clases de entidad* describen objetos pasivos. No inician las interacciones. Los objetos de entidad pueden participar en muchos casos de uso y normalmente sobreviven a las transiciones individuales. Se muestran como un círculo con una línea debajo.

Estereotipos de asociación

Existen dos estereotipos de asociación: comunicar y suscribir. La notación es una ruta de asociación con el nombre del estereotipo entre comillas.

Las *asociaciones de comunicación* conectan un actor con un caso de uso con el cual se comunica. Ésta es la única asociación posible entre actores y casos de uso, así que se puede omitir la palabra clave.

Las *asociaciones de suscripción* conectan la clase cliente (el suscriptor) con la clase proveedora (el editor). El suscriptor especifica un conjunto de eventos que pueden ser producidos por el editor. Cuando se produce uno de estos eventos, se le notifica al suscriptor.

Extensiones de modelado de negocios

Estas extensiones están destinadas a modelar organizaciones de negocios del mundo real y para comprender situaciones reales, más que para la implementación de software. Estos estereotipos no son necesarios para el modelado de negocios, pero abarcan algunas situaciones comunes.

Estas extensiones incluyen estereotipos de unidades de empaquetado, clases y asociaciones así como algunas restricciones relativas a la conexión de elementos. No hay etiquetas nuevas.

Estereotipos organizativos

La Tabla C-2 muestra los estereotipos organizativos correspondientes al modelado de procesos de negocios.

El modelo de caso de uso es un modelo de la vista de casos de uso. El sistema de casos de uso y el paquete de casos de uso son dos capas de organización de su contenido.

El modelo de objetos es un modelo de la estructura interna del sistema de negocios. El sistema objeto es su subsistema de más alto nivel, que contiene unidades organizativas y unidades de trabajo como capas inferiores. Una unidad organizativa corresponde a una unidad organizativa del negocio real, y una unidad de trabajo es un agrupamiento significativo, aun siendo de menor entidad.

No existen iconos especiales para la unidades organizativas del modelado de negocios; utilizan el símbolo de carpeta con un estereotipo entre comillas.

Tabla C-2 Estereotipos organizativos para el modelado de negocios

<i>Clase base</i>	<i>Captura de casos de uso</i>	<i>Modelo de Objetos</i>
modelo	modelo de casos de uso	modelo de objetos
paquete	sistema de caso de uso paquete de caso de uso	
subsistema		sistema de objetos unidad organizativa unidad de trabajo

Estereotipos de clase

Además de los actores (que están definidos en UML estándar), los objetos de negocios tiene varios sistemas de clase: trabajador, trabajador de caso, trabajador interno, entidad. Se muestran en la Figura C-3.

Un trabajador representa un ser humano que actúa en el interior del sistema. Un trabajador de caso es un trabajador que interacciona directamente con actores externos. Un trabajador interno es un trabajador que interacciona con trabajadores y entidades situadas dentro del sistema.

Las entidades de clase describen objetos pasivos. No inician las interacciones. Los objetos de entidad pueden participar en muchos casos de uso y normalmente sobreviven a interacciones individuales. En una situación de trabajo, las entidades suelen representar productos del trabajo.

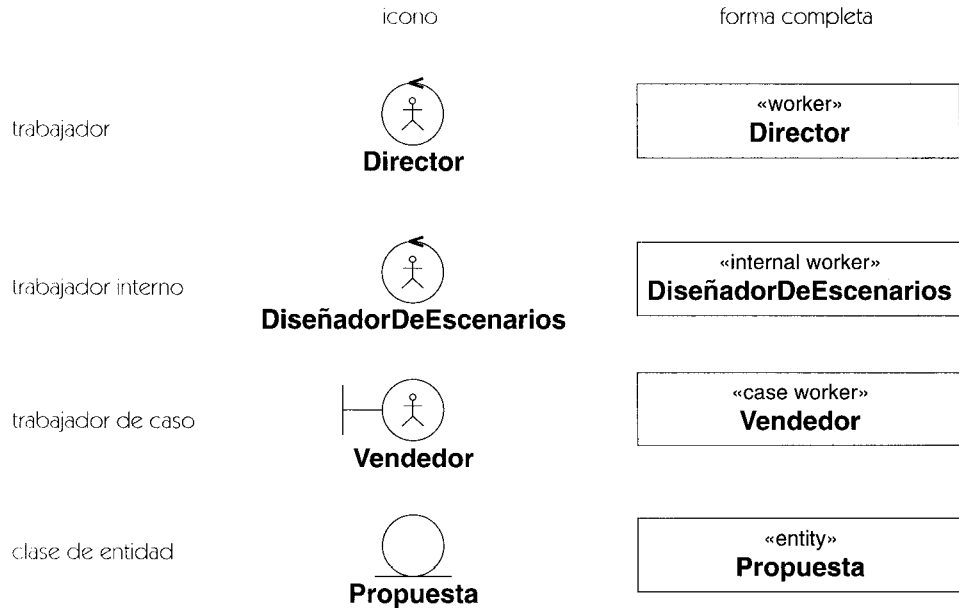


Figura C-3 Estereotipos de clase para el modelado de negocios

Estereotipos de asociación

Existen dos estereotipos de asociación: comunicar y suscribir. Son los mismos que para un proceso de desarrollo de software. La notación es una ruta de asociación con la palabra clave del estereotipo entre comillas.

Una asociación de comunicación conecta un actor con aquel caso de uso con el cual se comunica. Se trata de la única asociación existente entre actores y casos de uso, así que se puede omitir la palabra clave.

Una asociación de suscripción conecta una clase cliente (el suscriptor) con una clase proveedora (el editor). El suscriptor especifica un conjunto de eventos que puede producir el editor. Cuando se produce uno de estos eventos, se le comunica al suscriptor.



- [**Blaha-98**] Michael Blaha, William Premerlani. *Object-Oriented Modeling and Design for Database Applications*. Prentice Hall, Upper Saddle River, N.J., 1998.
- [**Booch-91**] Grady Booch. *Object-Oriented Analysis and Design with Applications*, 1.^a Edición Benjamin/Cummings, Redwood City, Calif., 1991.
- [**Booch-94**] Grady Booch. *Object-Oriented Analysis and Design with Applications*, 2.^a Edición Benjamin/Cummings, Redwood City, Calif., 1994.
- [**Booch-96a**] Grady Booch. *Object Solutions: Managing the Object-Oriented Project*. Addison-Wesley, Menlo Park, Calif., 1996.
- [**Booch-96b**] Grady Booch. *Best of Booch: Designing Strategies for Object Technology*. SIGS Books, Nueva York, N.Y., 1996.
- [**Booch-99**] Grady Booch, James Rumbaugh, Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, Reading, Mass., 1999.
- [**Buschmann-96**] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, Michael Stal. *Pattern-Oriented Software Architecture: A System of Patterns*. Wiley, Chichester, U.K., 1996.
- [**Coad-91**] Peter Coad, Edward Yourdon. *Object-Oriented Analysis*, 2.^a Edición Yourdon Press, Englewood Cliffs, N.J., 1991.
- [**Coleman-94**] Derek Coleman, Patrick Arnold, Stephanie Bodoff, Chris Dollin, Helena Gilchrist, Fiona Hayes, Paul Jeremaes. *Object-Oriented Development: The Fusion Method*. Prentice Hall, Englewood Cliffs, N.J., 1994.
- [**Cox-86**] Brad J. Cox. *Object-Oriented Programming.-An Evolutionary Approach*. Addison-Wesley, Reading, Mass., 1986.
- [**Embley-92**] Brian W. Embley, Barry D. Kurtz, Scott N. Woodfield. *Object-Oriented Systems Analysis: A Model-Driven Approach*. Yourdon Press, Englewood Cliffs, N.J., 1992.
- [**Gamma-95**] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, Mass., 1995.
- [**Goldberg-83**] Adele Goldberg, David Robson. *Smalltalk-80: The Language and Its Implementación*. Addison-Wesley, Reading, Mass., 1983.
- [**Harel-98**] David Harel, Michal Politi. *Modeling Reactive Systems With Statecharts: The STATEMATE Approach*. McGraw-Hill, Nueva York, N.Y., 1998.
- [**Jacobson-92**] Ivar Jacobson, Magnus Christerson, Patrik Jonsson, Gunnar Overgaard. *Object- Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, Wokingham, Inglaterra, 1992.
- [**Jacobson-95**] Ivar Jacobson, Maria Ericsson, Agneta Jacobson. *The Object Advantage: Business Process Reengineering with Object Technology*. Addison-Wesley, Wokingham, Inglaterra, 1995.

- [**Jacobson-97**] Ivar Jacobson, Martin Griss, Patrik Jonsson. *Software Reuse: Architecture, Process and Organization for Business Success*. Addison-Wesley, Harlow, Inglaterra, 1997.
- [**Jacobson-99**] Ivar Jacobson, Grady Booch, James Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, Reading, Mass., 1999.
- [**Martin-92**] James Martin, James Odell. *Object-Oriented Analysis and Design*. Prentice Hall, Englewood Cliffs, N.J., 1992.
- [**Meyer-88**] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, Nueva York, N.Y., 1988.
- [**Rumbaugh-91**] James Rumbaugh, Michael Blaha, William Premerlani, Frederick Eddy, William Lorensen. *Object-Oriented Modeling and Design*. Prentice Hall, Englewood Cliffs, N.J., 1991.
- [**Rumbaugh-96**] James Rumbaugh. *OMT Insights: Perspectives on Modeling from the Journal of Object-Oriented Technology*. SIGS Books, Nueva York, N.Y., 1996.
- [**Rumbaugh-99**] James Rumbaugh, Ivar Jacobson, Grady Booch. *The Unified Modeling Language Reference Manual*. Addison-Wesley, Reading, Mass., 1999.
- [**Selic-94**] Bran Selic, Garth Gullekson, Paul T. Ward. *Real-Time Object-Oriented Modeling*. Wiley, Nueva York, N.Y., 1994.
- [**Shlaer-88**] Sally Shlaer, Stephen J. Mellor. *Object-Oriented Systems Analysis: Modeling the World in Data*. Yourdon Press, Englewood Cliffs, N.J., 1988.
- [**Shlaer-92**] Sally Shlaer, Stephen J. Mellor. *Object Lifecycles: Modeling the World in States*. Yourdon Press, Englewood Cliffs, N.J., 1992.
- [**UML-98**] *Unified Modeling Language Specification*. Object Management Group, Framingham, Mass., 1998. Internet: www.omg.org.
- [**Ward-85**] Paul Ward, Stephen J. Mellor. *Structured Development for Real-Time Systems: Introduction and Tools*. Yourdon Press, Englewood Cliffs, N.J., 1985.
- [**Warmer-99**] Jos B. Warmer, Anneke G. Kleppe. *The Object Restriction Language: Precise Modeling with UML*. Addison-Wesley, Reading, Mass., 1999.
- [**Wirfs-Brock-90**] Rebecca Wirfs-Brock, Brian Wilkerson, Lauren Wiener. *Designing Object-Oriented Software*. Prentice Hall, Englewood Cliffs, N.J., 1990.
- [**Yourdon-79**] Edward Yourdon, Larry L. Constantine. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Yourdon Press, Englewood Cliffs, N.J., 1979.



Las referencias principales aparecen en negrita.

A

abstracción, **103**
abstracto/a, **104**
acceder, 89
acceso, 89, **107**
access, **479**
acción, 64, **110**
 asíncrona, **113**
 de entrada, 65, 66, **113**
 de salida, 65, 66, **114**
 síncrona, **115**
activación, 77, **115**
actividad, **117**
activo, **118**
actor, 56, **120**
agregación, **122**
 compuesta, **125**
agregado, **125**
alcance, **125**
 de destino, **127**
 de propietario, **127**
 original, **128**
amigo, **128**
análisis, **129**
 estructurado, 4
antecesor, **129**
argumento, **129**
 formal, **130**
Arnold, Patrick, 517
arquitectura, **130**
artefacto, **131**

asociación, 42, 44, **131**
 binaria, **135**
 clase asociación, 43
 de comunicación, **136**
 n-aria, **136**
association, **479**
atómico, **138**
atributo, **139**
autotransición, **143**

B

become (se convierte en), 80, **143**, **479**
bidireccionalidad, 45
bien formado, 53, 96, 99, **145**
bifurcación, **145**
bind (ligar), **147**, **480**
Blaha, Michael, 4, 517-518
Bock, Conrad, 6
Bodoff, Stephanie, 517
Booch, Grady, 4-6, 517-518
booleano/a, **147**
Brodsky, Steve, 6
Buschmann, Frank, 517

C

C++, 4
cadena, **147**
calificador, **148**
call, **480**
calle, 72, **154**

capa, **156**
 característica, **156**
 de comportamiento, **156**
 estructural, **156**
 cardinalidad, **156**
 caso de uso, **157**
 Cheesman, John, 6
 Christerson, Magnus, 518
 clase, 38, 40, **162**
 abstracta, **167**
 activa, **167**
 asociación, 43, **168**
 compuesta, **170**
 de implementación, **170**
 directa, **171**
 clase-en-un-estado, **171**
 clasificación
 dinámica, 48, 96, **173**
 estática, 8, 22, **37**, 48, 54, 96, **174**
 y dinámica, 48
 múltiple, 48, **174**
 simple, 48, **174**
 y múltiple, 48
 clasificador, 38, **174**
 cliente, **175**
 CLOS, 4
 Coad, Peter, 4, 517
 Cobol, 4
 colaboración, 75, **175**
 realización de casos de uso, 57
 Coleman, Derek, 5, 517
 combinación, **182**
 comentario, **183**
 comillas, **184**
 compartimento, **184**
 compendio, **185**
 complete, **480**
 componente, 83, 84, **186**
 comportamiento, **191**
 composición, **191**
 concreto, **197**
 concurrencia, **197**
 dinámica, **198**
 condición de guarda, 64, **198**
 configuración del estado activo, **198**
 conflicto, **199**
 conjunción; véase unión
 consideraciones de lenguaje de programación, 97
 Constantine, Larry, 4, 518

construcción, **199**
 constructor, **199**
 consulta, **199**
 contenedor, **200**
 contexto, **200**
 Cook, Steve, 6
 copia, **200**
 copy, **480**
 Cox, Brad, 4, 517
 CRC, 5
 creación, **201**
 create, **481**

D

D'Souza, Desmond, 6
 delegación, **203**
 DeMarco, Tom, 4
 dependencia, 50, **203**
 entre paquetes, 88
 tabla de dependencias, 51
 derivación, **205**
 derive, **481**
 desarrollo
 estereotipos, tabla de proceso, 512
 extensiones de proceso, **511**
 incremental, **206**
 iterativo, **206**
 método tradicional, 4
 métodos de orientado a objetos, 4
 descendiente, **206**
 descriptor, **206**
 completo, **207**
 Desfray, Philippe, 6
 DeSilva, Dilhar, 6
 despliegue, **207**
 destroy, **481**
 destroyed, **482**
 destrucción, **208**
 destruir, **209**
 diagrama, **209**
 de actividad, 28, 71, 72, 73, **210**
 de casos de uso, 24, 55, **210**
 de clases, 23, **211**
 de colaboración, 26, 78, **211**
 de componentes, 30, **211**
 de despliegue, 31, 32, 85, **211**
 de estados, 27, **214**
 de interacción, **214**

- de objetos, 54, **215**
- de secuencia, 25, 76, **216**
- Digre, Tom, 6
- discriminador, **219**
- diseño, **221**
- diseño estructurado en tiempo real, 4
- disjoint, **482**
- disparador, **221**
- disparar, **222**
- división, **223**
- document, **482**
- documentation, **482**
- Dollin, Chris, 517

E

- Eddy, Frederic, 4, 518
- eficiencia de navegación, **224**
- Eiffel, 4
- ejecutar hasta finalizar, **225**
- elaboración, **226**
- elemento, **226**
 - de representación, **228**
 - derivado, **228**
 - generalizable, **230**
 - ligado, 231
 - modelo, **227**
 - parametrizado, **233**; *véase* plantilla
- Embley, Brian, 517
- emisor, **233**
- enlace, **233**
 - transitorio, **235**
- entorno, **95**
- enumeración, **236**
- enumeration, **482**
- enviar, **237**
- Ericsson, Maria, 518
- escenario, **240**
- espacio de nombres, **241**
- especialización, **241**
- especificación, **242**
- especificador de interfaz, **242**
- estado, 62, **243**
 - abreviado externo, **249**
 - compuesto, 66, **250**
 - concurrente, 69
 - de acción, **252**
 - de actividad, **253**
 - de flujo de objeto, **254**

- de historia, **257**
- de origen, **259**
- de referencia a submáquina, **259**
- de sincronización, **261**
- de unión o conjunción, **265**
- destino, **249**
- final, **266**
- inicial, **268**
- simple, **270**
- tabla de estados, 67
- estereotipo, 93, 94, 97, **270**
- estereotipos de modelado de negocios, tabla de, 514
- etiqueta, **273**
- evento, 60, **274**
 - actual, **275**
 - de cambio, 61, **276**
 - de llamada, 61, **278**
 - de señal, **279**
 - diferido, **279**
 - disparador, 63
 - temporal, 62, **280**
- excepción, **281**
- executable, **483**
- exportar, **282**
- expresión, **282**
 - booleana, **283**
 - de acción, **283**
 - de actividad, **283**
 - de conjunto de objetos, **284**
 - de iteración, **284**
 - de procedimiento, **285**
 - de tipo, **286**
 - temporal, **286**
- extend, **483**
- extender, **287**
- extensión, **292**
- extensiones
 - de modelado de negocios, **513**
 - de proceso, **511**
 - de desarrollo de software, **511**
- extremo de asociación, **292**
- Eykholt, Ed., 6

F

- facade, **483**
- fachada, **483**
- fase de elaboración, **226**
- fases de modelado, **295**

file, **483**
 filtrado de listas, 341
 fin de un enlace, **297**
 flujo, 79, **297**
 de objeto, **297, 298**
 foco de control, **299**
 Fortran, 4
 framework, **483**
 friend, **483**
 frozen, 474
 fusión, 5

G

Gamma, Erich, 517
 generación de código, 97
 generalización, 45, **299**
 de asociaciones, **302**
 de casos de uso, **304**
 Gery, Eran, 6
 Gilchrist, Helena, 517
 global, **484**
 Goldberg, Adele, 4, 518
 grafo de actividades, **305**
 Griss, Martin, 6, 518
 Gullekson, Garth, 518

H

Harel, David, 6, 518
 Hayes, Fiona, 517
 Helm, Richard, 517
 herencia, 47, **312**
 de implementación, **313**
 de interfaz, **314**
 múltiple, 47, **314**
 privada, **314**
 simple, **315**
 herramienta de modelado, 98
 hijo/a, **315**
 hilo, **315**
 condicional, **315**
 hipervínculo, **316**
 Hogg, John, 6
 hoja, **316**

I

iconos de control, **317**
 identidad, **320**

implementación, **320**
 implementation, **484**
 implementation class, **484**
 implicit, **484**
 import, **485**
 importación, 89
 importar, **321**
 inactivo, **321**
 include, **485**
 incluire, **321**
 incomplete, **485**
 información de fondo, **323**
 ingeniería inversa, 97
 iniciación, **324**
 inicio, **324**
 instance of, **485**
 instancia, **325**
 directa, **329**
 indirecta, **329**
 válida de un sistema, 53
 instancia de, 52, **329**
 caso de uso, **329**
 instanciable, **327**
 instanciación, **327**
 instanciar, **329**
 instantánea, 54, **329**
 instantiate, **485**
 intención, **329**
 intento, 16
 interacción, 76, **330**
 interfaz, 49, **331**
 invariant, **486**
 invariante, **334**
 Iyengar, Shridar, 6

J

Jacobson, Agneta, 518
 Jacobson, Ivar, 6-7, 517-518
 Jeremaes, Paul, 517
 Johnson, Patrick 518
 Johnson, Ralph 517

K

Khalsa, G.K., 6
 Kleppe, Anneke, 518
 Kobryn, Cris, 6
 Kurtz, Barry, 517

L

leaf, **486**
 Lenguaje de Restricción de Objetos; *véase* OCL
 lenguaje de restricciones, **52**
 Lenguaje Unificado de Modelado; *véase* UML
 library, **486**
 libros sobre orientación a objetos, **4**
 ligadura, **335**
 línea de vida, **337**
 del objeto, **338**
 Liskov, Barbara, **46**
 lista, **338**
 de parámetros, **341**
 de propiedades, **341**
 llamada, **343**
 local, **486**
 localización, **342**
 location, **486**
 Lorensen, William, **4**, **518**

M

mal formado (modelo), **96**, **344**
 máquina de estados, **59**, **68**, **69**, **345**
 vista de, **27**, **59**
 marca de tiempo, **353**
 marcador, **354**
 gráfico, **354**
 marco de trabajo, **354**
 Martin, James, **518**
 materialización, **354**
 materializar, **355**
 mecanismos de extensión, **9**, **91**, **511**
 Mellor, Stephen, **4**, **518**
 mensaje, **79**, **355**
 metacalse, **362**
 metaclass, **487**
 meta-modelado, **362**
 metamodelo, **95**, **362**
 de UML, **495**, **496**
 metaobjeto, **362**
 metarelación, **363**
 método, **363**
 de Booch, **5**, **7**
 Meunier, Regine, **517**
 Meyer, Bertrand, **4**, **518**
 miembro, **364**
 Miller, Joaquin, **6**

modelado, **11**
 modelo, **89**, **364**
 de casos de uso, **365**
 definición, **11**
 inconsistente, **96**, **98**
 niveles de un, **13**
 propósito, **11**
 significado, **16**
 módulo, **366**
 muchos, **366**
 multiobjeto, **366**
 multiplicidad, **367**
 de asociación, **369**
 de atributo, **370**
 de clase, **371**

N

navegabilidad, **371**
 navegable, **373**
 navegación, **373**
 new, **487**
 no interpretado, **375**
 nodo, **84**, **373**
 nombre, **375**
 de clase, **376**
 de rol, **377**
 de ruta, **379**
 nota, **379**
 notación
 canónica, **96**, **97**, **380**
 resumen, **499**
 número de secuencia, **381**

O

Object Management Group, **5**, **495**, **518**
 Objective C, **4**
 Objectory, **5**, **7**, **511**
 objeto, **381**
 activo, **384**
 compuesto, **385**
 pasivo, **387**
 persistente, **387**
 transitorio, **387**
 OCL, **52**, **387**, **495**
 Odell, James, **6**, **518**
 OMG; *véase* Object Management Group
 OMT, **5**, **7**
 operación, **389**

abstracta, **394**
ordenación, **396**
orientación a objetos, historia de la, 4
Övergaard, Gunnar, 6, 518
overlapping, **487**

P

padre, **398**
palabra clave, **398**
Palmkvist, Karin, 6
paquete, 87, 88, **399**
 de fundamentos, 496
parameter, **487**
parámetro, **402**
 real, **404**; véase argumento
participantes, **404**
patrón, 80, **404**
perfil, 94
permiso, **406**
persistence, **487**
plantilla, **406**
polimórfico/a, 46, **410**
Politi, Michal, 518
postcondición, **412**
postcondition, **488**
powertype, **488**
precondición, **413**
precondition, **488**
Premerlani, William, 4, 517-518
principio de capacidad de sustitución, 46, **414**
privado, **415**
proceso de desarrollo, **415**
proceso/procesar, **417**
process, **488**
producto, **417**
propiedades, **417**
protegida, **417**
proveedor, **418**
proyección, **418**
pseudoatributo, **418**
pseudoestado, **418**
pública, **419**
punto
 de extensión, **419**
 de variación semántica, **421**

Q

query, 393

R

Ramackers, Guus, 6
Rational Software Corporation, 5
realización, 48, **421**
 comparada con la generalización, 49
realizar, **423**
recepción, **423**
receptor, **424**
recibir, **424**
Reenskaug, Trygve, 6
referencia, **424**
refinamiento, 50, **425**
refine, **427**, **488**
reglas de visibilidad, 89
relación, 41, **427**
 tabla, 41
relaciones de caso de uso, 57, 58
repositorio, **428**
requirement, **489**
requisito, **428**
responsabilidad, **428**
responsibility, **489**
restricción, 52, 53, 91, 92, **429**
reunificación, **432**
reutilización, **433**
Robson, David, 4, 518
Rohner, Hans, 517
rol, **434**
 de asociación, **434**
 de clasificador, **435**
 de colaboración, **436**
Rumbaugh, James, 4-6, 517-518
ruta, **438**

S

secuencia de acciones, **440**
Seidewitz, Ed., 6
self, **489**
Selic, Bran, 6, 518
semántica, **440**
semantics, **489**
send, **489**
señal, 60, **440**
 declaración, 60
Shlaer, Sally, 4, 518
Short, Keith, 6
signatura, **443**

Simula-67, 4
 sintaxis del libro, 10
 sistema, **443**
 Smalltalk, 4
 sociedad de objetos cooperantes, 75
 solicitud, **443**
 Sommerland, Peter, 517
 Stal, Michael, 517
 stereotype, **490**
 stub, **490**
 subclase, **443**
 subestado, **444**
 concurrente, **444**
 disjunto, **444**
 ortogonal, **444**
 submáquina, 69, **444**
 subsistema, 90, **444**
 subtipo, **445**
 superclase, **446**
 supertipo, **447**
 supratipo, **447**
 system, **490**

T

tabla
 acciones, 65
 clasificadores, 39
 de estados, 67
 dependencias, 51
 estereotipos de proceso de desarrollo, 512
 estereotipos del modelo de negocios, 515
 eventos, 60
 flujos, 80
 relaciones, 41
 entre casos de uso, 57
 entre vistas, 35
 transiciones, 63
 vistas y diagramas, 22
 table, **490**
 thread, **491**
 tiempo, **447**
 absoluto, 286
 de análisis, **448**
 de compilación, **448**
 de diseño, **448**
 de ejecución, **448**
 de modelado, **448**

 de transición, **448**
 tipo, **448**
 de dato, **450**
 del lenguaje, **451**
 primitivo, **451**
 trace, **491**
 transición, 62, 63, **452**
 abreviada, **456**
 compleja, **457**
 de finalización, 64, **463**
 externa, 62
 fase, **464**
 interna, 66, **464**
 simple, **466**
 sin disparador, **466**
 transient, **491**
 traza, 50, **466**
 tupla, **467**
 type, **491**

U

UML
 adaptación de, 94
 áreas conceptuales, 8
 definición, 3
 documentos de especificación, 495
 entorno, **95**
 estandarización, 5
 extensiones de proceso, **511**
 historia, 4
 metamodelo, **495**, 496
 objetivos, 7
 resumen de la notación, **499**
 vistas, 21
 tabla de, 22
 unidad de distribución, **467**
 unificado (definición de), 6
 unión, **467**
 use, **468**, **491**
 uso, 50, **468**
 de la tipografía, **469**
 utilidad, **469**
 utility, **492**

V

valor, **470**
 dato, **470**

etiquetado, 92, 93, 97, **470**
 inicial, **472**
 no especificado, 99, **473**
 nulo, 99
 por defecto, **473**
 prefijado, 16
 variabilidad, **473**
 vértice, **474**
 visibilidad, **474**
 vista, **476**
 conexiones entre vistas, 34
 de actividades, 29, 71, **476**
 de administración del modelo, **476**
 de casos de uso, 24, **55**, **476**
 de comportamiento, **476**
 de despliegue, 30, **476**
 de implementación, 29, 83, **477**
 de interacción, 25, **75**, **477**
 de máquina de estados, **477**
 de un modelo, **87**
 dinámica, 53, **477**
 estática, 8, 22, **37**, 54, **477**
 estructural, **478**

funcional, **478**
 resumen, 21
 tabla de vistas, 22
 vistas físicas, 29, **83**
 Vlissides, John, 517

W

Ward, Paul, 4, 518
 Warmer, Jos, 6, 518
 Wiegert, Oliver, 6
 Wiener, Lauren, 4, 518
 Wilkerson, Brian, 4, 518
 Wirfs-Brock, Rebecca, 4, 518
 Woodfield, Scott, 517

X

xor, **492**

Y

Yourdon, Edward, 4, 517-518